

ACME-NMS · REBUILD PLAN

แผนการสร้าง acme-nms ใหม่ทีละขั้น

ตั้งแต่ engine ไปจนถึงส่งมอบ appliance ให้ลูกค้า แบ่งเป็น 5 phase พร้อมขั้นตอน ไฟล์ วิธีตรวจว่าผ่าน และข้อผิดพลาดที่พบบ่อย

ฉบับวันที่ 7 ตุลาคม 2026

อ้างอิงข้อกำหนดจาก CLAUDE.md (§1–§10) ของโปรเจกต์

Stack: Zabbix 7.0.29 · TimescaleDB 2.28.3 (PG16) · Redis · FastAPI · React

สารบัญ

1. ภาพรวมระบบและกฎที่ห้ามละเมิด
2. สถานะ ณ วันที่ 7 ตุลาคม 2026
3. Phase 0: ประกอบ engine และทำความเข้าใจ Zabbix
4. Phase 1: Backend core
5. Phase 2: Provisioning และอุปกรณ์จริง
6. Phase 3: Frontend
7. Phase 4: ส่งมอบ appliance
8. Decision ที่ยังเปิดอยู่
9. ภาคผนวก A: คำสั่ง re-verify ข้อเท็จจริงของ Zabbix
10. ภาคผนวก B: `ops/check-boundary.sh`
11. ภาคผนวก C: `lab/ndjson-sink/sink.py`

วิธีอ่านเอกสารนี้

แต่ละขั้นตอนมีหัวข้อเหมือนกันหมด:

- เป้าหมาย: ทำขั้นนี้ไปเพื่ออะไร
- ไฟล์: path ตาม S6 ของ `CLAUDE.md`
- ขั้นตอน: สิ่งที่ต้องทำ เรียงตามลำดับ
- ตรวจสอบว่าผ่าน: คำสั่งและผลที่ควรได้
- ข้อผิดพลาดที่พบบ่อย

เรื่องที่ขึ้นกับ version: สิ่งเกี่ยวกับพฤติกรรมของ Zabbix ซึ่งอาจเปลี่ยนตาม patch release (เช่น enum ของ API หรือชื่อ field ใน payload) จะมีคำเตือนกำกับไว้ ให้ยืนยันกับ instance ของเราเองก่อนใช้ทุกครั้ง

1. ภาพรวมระบบและกฎที่ห้ามละเมิด

1.1 บริบท

ทีมของเราทำ 2 อย่างที่ site ลูกค้า:

- ติดตั้ง network: Cisco core switch, Cisco access switch และ Aruba AP
- ส่งมอบระบบ monitoring ที่เป็น **product** ของเราเอง ชื่อ `acme-nms`

Zabbix เป็น engine ที่อยู่ข้างใต้และซ่อนจากลูกค้า backend กับ frontend ของเราเป็น closed-source ส่วน Zabbix ใช้ image ทางเราโดยไม่แก้ไขอะไรเลย

1.2 ภาพรวมการไหลของข้อมูล



ทาง	รับส่งอะไร	โค้ดฝั่งเรา
JSON-RPC API (pull)	configuration, provisioning, CRUD ของ host/template/item/trigger, lookup ตามต้องการ	backend/app/zabbix/client.py
Connector (push, NDJSON ผ่าน HTTP POST)	item value และ event ทั้งหมด	backend/app/api/hooks.py → store ของเรา

1.3 กฎที่ห้ามละเมิด

กฎข้อมูล (§3):

- กราฟ dashboard และ SSE อ่านจาก store ของเราเท่านั้น ห้ามใช้ `history.get`
- Store ของเราต้องเป็น DB แยก หรืออย่างน้อยเป็น schema แยกที่มี role ของตัวเอง
- Backend ห้ามอ่าน table ของ Zabbix

กฎ boundary (§4):

- ใน `domain/`, `services/`, `api/`, `auth/` และ `frontend/` ห้ามมีคำว่า `zabbix` ไม่ว่าจะเป็นตัวพิมพ์เล็กหรือใหญ่
- ห้ามมีชื่อ field ของ engine: `eventid`, `hostid`, `itemid`, `triggerid`, `clock`, `ns`
- ID ที่ส่งข้าม boundary ต้องเป็น opaque string คือ `device_ref` และ `problem_ref`
- มีแค่ 2 ที่ที่ได้รับยกเว้น: `backend/app/zabbix/**` และ `backend/app/container.py`

กฎแหล่งที่มาของโค้ด (§5):

- ห้าม `git clone` และห้าม copy โค้ดจาก repo, gist หรือ blog
- ห้ามดาวน์โหลดไฟล์ `compose` ของ `zabbix-docker`
- เขียนเองทุกบรรทัดให้อธิบายได้ ส่วน dependency ใช้ `pip` หรือ `npm` ได้ตามปกติ

License (§7):

- Zabbix 7.0 เป็น AGPLv3 ห้ามแก้ไข source ของ Zabbix
- ห้ามเขียน frontend module ของ Zabbix แล้วปิด source
- ตอนส่งมอบต้องแนบ LICENSE ของ Zabbix และ written offer for source

1.4 หลักการทำงาน (§9)

- ทดสอบกับอุปกรณ์ 1 ตัวก่อน แล้วค่อยขยายไปทั้งหมด

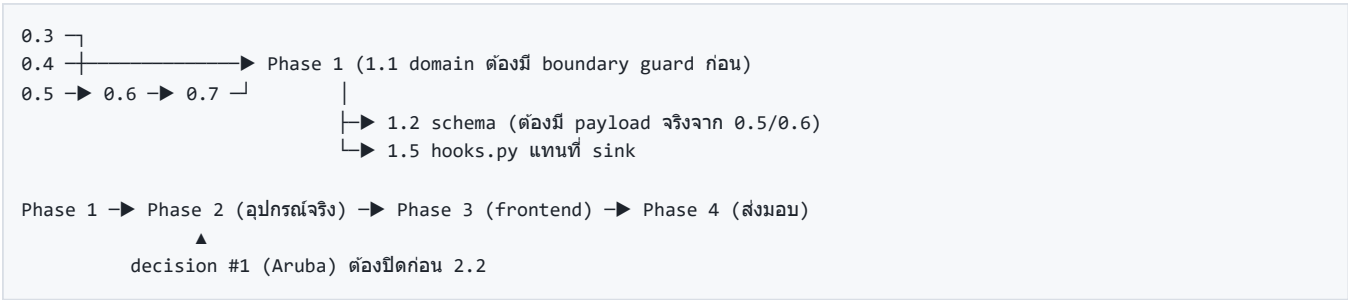
- ตรวจสอบค่าที่ได้เทียบกับ **CLI** จริงเสมอ ห้ามเชื่อ template
- SNMPv3 ใช้ authPriv และ counter แบบ 64-bit (ifHCInOctets) ส่วน throughput ให้ poll ทุก 60 วินาที
- ห้ามเก็บ credential ใน git
- SNMPv3 credential เก็บใน Vault macro หรือ Secret text macro เท่านั้น
- ใช้ dependent trigger เพื่อกันไม่ให้ core ล่มแล้วเกิด alert ปลอมจากอุปกรณ์ปลายทาง N ตัว

2. สถานะ ณ วันที่ 7 ตุลาคม 2026

ชั้น	งาน	สถานะ
0.1	Engine: db + server + web (engine/docker-compose.yml)	เสร็จ: ทั้ง 3 ตัวขึ้น healthy
0.2	API client + smoke test	เสร็จ: smoke ผ่านครบ 3 บรรทัด
0.3	backend/.env.example	ไฟล์หาย ต้องสร้างใหม่
0.4	ops/check-boundary.sh + hook	ยังไม่เริ่ม
0.5	Connector → sink (values)	ยังไม่เริ่ม
0.6	Event probe	ยังไม่เริ่ม
0.7-0.9	เอกสาร, git, re-verify	ยังไม่เริ่ม
Phase 1-4	ทุกงาน	ยังไม่เริ่ม

เรื่องค้างที่ต้องจัดการ: admin API token provisioning-admin เคยแสดงออกมาแบบเต็มใน output ตอนตรวจไฟล์ ควร **Regenerate** ที่ Users → API tokens แล้วอัปเดต backend/.env จากนั้นรัน smoke test อีกรอบ

ลำดับงานที่ต้องพึ่งกัน



Phase 3 เริ่มคู่ขนานกับ Phase 2 ได้ ทันทีที่ read API ใน 1.6 คงที่แล้ว

3. Phase 0: ประกอบ engine และทำความเข้าใจ Zabbix

เป้าหมายของ phase: พิสูจน์ทั้ง 2 ทาง (API และ Connector) กับ instance ของเราเอง และเก็บ payload จริงไว้ ก่อนเขียนโค้ด product สักบรรทัด

เงื่อนไขปิด phase:

- ตอบได้จากไฟล์จริงว่า payload ของ value และ event มี field อะไรบ้าง
- รันคำสั่งในภาคผนวก A ผ่านทุกข้อ
- โปรเจกต์เป็น git repo แล้ว และไม่มี secret หลุดเข้าไป

0.1 Engine: TimescaleDB + Zabbix server + Zabbix web ✓ เสร็จ

เป้าหมาย ยก Zabbix 7.0 ขึ้นมาจาก image ทาง docker โดยเขียน compose เองทั้งไฟล์

ไฟล์ engine/docker-compose.yml, engine/.env.example, engine/.env (ห้าม commit)

สิ่งที่ทำไปแล้ว

- Pin tag แบบเดิม: timescale/timescaledb:2.28.3-pg16, zabbix/zabbix-server-pgsql:alpine-7.0.29 และ zabbix/zabbix-web-nginx-pgsql:alpine-7.0.29
- DB ไม่เปิด port ออกนอก network ส่วน web bind ที่ 127.0.0.1:8080 เท่านั้น
- ตั้ง ENABLE_TIMESCALEDB=true (มีผลแค่ตอนสร้าง DB ครั้งแรก) และ ZBX_STARTCONNECTORS=2

ตรวจว่าผ่าน

```
docker compose -f engine/docker-compose.yml ps # ทั้ง 3 ตัวต้องเป็น (healthy)
curl -s -X POST http://127.0.0.1:8080/api_jsonrpc.php -H 'Content-Type: application/json-rpc' \
-d '{"jsonrpc": "2.0", "method": "apiinfo.version", "params": {}, "id": 1}'
# expect: {"jsonrpc": "2.0", "result": "7.0.29", "id": 1}
```

ข้อผิดพลาดที่พบบ่อย

- แก้ POSTGRES_PASSWORD ใน .env หลังจาก volume ถูกสร้างแล้ว รหัสใน DB จะไม่เปลี่ยนตาม
- เปลี่ยน tag ของ TimescaleDB โดยไม่เช็คว่าอยู่ในช่วงที่ Zabbix version นั้นรองรับ (ดู §7)
- ใช้ tag latest แบบนี้ผลลัพธ์จะไม่คงที่

0.2 API client + smoke test ✓ เสร็จ

เป้าหมาย เขียน JSON-RPC client เองสำหรับฝั่ง configuration

ไฟล์ backend/app/zabbix/client.py, backend/app/zabbix/smoke.py, backend/app/config.py, backend/pyproject.toml

หลักการที่ต้องรักษาไว้

- token เป็น argument บังคับของทุก call ไม่อ่านจาก config.py (decision #4) ถ้าส่ง None หมายความว่า method นั้นไม่ต้อง auth และต้องเขียนออกมาให้เห็นชัด
- ชื่อ setting ขึ้นต้นด้วย ENGINE_* ห้ามใช้ ZABBIX_*
- Smoke test มี negative test ด้วย: token มั่วต้องถูกปฏิเสธ

ตรวจว่าผ่าน

```
cd backend
.venv\Scripts\python -m app.zabbix.smoke
# expect:
# version: 7.0.29
# hosts: ['Zabbix server']
# bad token rejected: host.get: [-32602] Invalid params. Not authorized.
```

ข้อผิดพลาดที่พบบ่อย

- รันนอกโฟลเดอร์ backend/ จะอ่าน .env ไม่เจอ แล้วได้ engine_token Field required
- Token มีช่องว่างหรือเครื่องหมายคำพูดติดมา หรือวาง token ไว้คนละบรรทัดกับ ENGINE_TOKEN=
- ลืมรหัส Admin: reset ผ่าน DB ด้วย bcrypt hash ได้ (ทำไปแล้วครั้งหนึ่ง) ห้ามลบ volume เพราะข้อมูลทั้งหมดจะหาย

0.3 สร้าง backend/.env.example ใหม่ □ ต้องทำ

เป้าหมาย มี template ไว้ commit โดยไม่มี secret อยู่ข้างใน

ไฟล์ backend/.env.example

```
# backend/.env.example – copy to backend/.env and fill in. Never commit .env.

ENGINE_URL=http://127.0.0.1:8080
# Provisioning/admin token only. Users -> API tokens in the engine UI.
ENGINE_TOKEN=
```

ตรวจว่าผ่าน

```
grep -c '^ENGINE_TOKEN=$' backend/.env.example # expect: 1
```

ข้อผิดพลาดที่พบบ่อย

- Rename .env.example เป็น .env จนไฟล์ template หายไป ต้อง **copy** ไม่ใช่ rename
- วาง token จริงลงใน .env.example

0.4 Boundary guard: ops/check-boundary.sh □ ต้องทำ

เป้าหมาย บังคับกฎ §4 ให้ทำงานอัตโนมัติก่อนสร้างไฟล์แรกใน guarded path

ไฟล์ ops/check-boundary.sh (โค้ดเดิมอยู่ในภาคผนวก B), .gitattributes, .claude/settings.json

ขั้นตอน

1. สร้าง ops/check-boundary.sh ตามภาคผนวก B และบันทึกเป็น LF
2. สร้าง .gitattributes ที่ root โดยใส่บรรทัด *.sh text eol=lf
3. แก้ hook ใน .claude/settings.json เป็น "command": "bash \"\$CLAUDE_PROJECT_DIR/ops/check-boundary.sh\""

Script ตรวจสอบอะไรบ้าง

กฎ	วิธี match	ตัวอย่างที่จับได้
ชื่อ engine	substring, ไม่สนใจตัวพิมพ์เล็กใหญ่	ZabbixClient, ZABBIX_URL
ID ของ engine	นับ _ เป็นตัวคั่น และรองรับรูปพหูพจน์	device_hostid, itemids
field สั้น	whole word (-w)	clock, ns แต่ไม่จับ clockwise, dns

ตรวจว่าผ่าน (รันจาก Git Bash ที่ root)

```
bash ops/check-boundary.sh; echo "exit=$?"
# expect: check-boundary: no guarded paths exist yet – OK / exit=0

mkdir -p backend/app/domain
printf 'device_hostid = 1\nfrom zabbix import x\nclock = 0\n' > backend/app/domain/_probe.py
bash ops/check-boundary.sh; echo "exit=$?"
# expect: BOUNDARY VIOLATION ครบ 3 กฎ / exit=2

printf 'device_ref = "a"\nclockwise = 1\ndns = 2\n' > backend/app/domain/_probe.py
bash ops/check-boundary.sh; echo "exit=$?"
# expect: check-boundary: OK (backend/app/domain) / exit=0

rm backend/app/domain/_probe.py && rmdir backend/app/domain
```

ข้อผิดพลาดที่พบบ่อย

- บันทึกเป็น CRLF แล้วเจอ \$'\n': command not found
- ใช้ exit 1 แทน exit 2 ทำให้ Claude Code มองไม่เห็นว่ามีข้อผิดพลาด
- ใช้ grep -w hostid จะจับ device_hostid ไม่ได้
- ใส่ container.py หรือ zabbix/ ลงใน GUARDED ทั้งที่ได้รับความนิยม
- เพิ่มข้อยกเว้นเพื่อหลบ false positive ที่ถูกต้องตาม spec ให้เปลี่ยนค่าในโค้ดแทน

0.5 Connector → sink: พิสูจน์ทาง push ของ item value □ ต้องทำ

เป้าหมาย ให้ Zabbix push value มายังตัวรับชั่วคราว แล้วเก็บ payload จริงลงดิสก์

ไฟล์ lab/ndjson-sink/sink.py (โค้ดเดิมอยู่ในภาคผนวก C), lab/ndjson-sink/.sink-token (ห้าม commit), lab/ndjson-sink/captured/

ขั้นตอน

1. สร้าง sink.py ตามภาคผนวก C ซึ่งใช้แค่ stdlib
2. รัน sink เป็น container ใน network netwatch-engine :

```
python -c "import secrets; print(secrets.token_urlsafe(32))" > lab/ndjson-sink/.sink-token
SINK_TOKEN=$(cat lab/ndjson-sink/.sink-token)

MSYS_NO_PATHCONV=1 docker run -d --name netwatch-sink \
  --network netwatch-engine \
  -e SINK_TOKEN="$SINK_TOKEN" \
  -v "$(pwd -W)/lab/ndjson-sink:/app" \
  -w /app \
  python:3.12-alpine python -u sink.py
```

3. สร้าง Connector 2 ตัวผ่าน API:

```
ENGINE_TOKEN=$(grep '^ENGINE_TOKEN=' backend/.env | cut -d= -f2 | tr -d '\r')
api() {
  curl -s -X POST http://127.0.0.1:8080/api_jsonrpc.php \
    -H 'Content-Type: application/json-rpc' \
    -H "Authorization: Bearer $ENGINE_TOKEN" -d "$1"; echo
}

api '{"jsonrpc":"2.0","method":"connector.create","id":1,"params":[
  {"name":"lab-sink-values","url":"http://netwatch-sink:8099/values",
    "data_type":0,"authtype":5,"token":"'$SINK_TOKEN'", "timeout":"10s"},
  {"name":"lab-sink-events","url":"http://netwatch-sink:8099/events",
    "data_type":1,"authtype":5,"token":"'$SINK_TOKEN'", "timeout":"10s"}
]}'

docker compose -f engine/docker-compose.yml exec zabbix-server \
  zabbix_server -R config_cache_reload
```

ขึ้นกับ **version**: data_type 0 คือ item values, 1 คือ events ส่วน authtype 5 คือ Bearer ค่าเหล่านี้อ้างอิงจาก Zabbix 7.0 ให้ตรวจกับ instance ของเราด้วย connector.get และ "output":"extend"

ตรวจว่าผ่าน

```
docker compose -f engine/docker-compose.yml top zabbix-server | grep -i connector
# expect: connector manager #1, connector worker #1, connector worker #2

docker logs -f netwatch-sink
# expect (ภายใน 1-2 นาที): [values] batch 1: N lines, ... Content-Type=application/x-ndjson

head -n 1 lab/ndjson-sink/captured/values/0001.ndjson | python -m json.tool
```

ข้อผิดพลาดที่พบบ่อย

- ตั้ง URL เป็น localhost ซึ่งจาก container ของ server จะหมายถึงตัว server เอง
- ลืม MSYS_NO_PATHCONV=1 หรือ pwd -W ทำให้ mount ว่าง และโฟลเดอร์ captured/ ไม่โผล่บนเครื่อง
- Log ของ server ขึ้น Couldn't resolve host แปลว่าอยู่คนละ network
- Log ของ sink ขึ้นบรรทัด 401 แปลว่า token ไม่ตรงกัน
- ZBX_STARTCONNECTORS=0 ทำให้สร้าง connector ได้ แต่ไม่มีอะไรถูกส่งออกไปเลย

0.6 Event probe: ทำให้เกิด problem และ recovery □ ต้องทำ

เป้าหมาย สร้าง "ปุ่ม" ไว้ทำให้เกิด event จริงได้ตลอด เพื่อเก็บ payload ของ event และใช้ทดสอบ pipeline ทั้งเส้นในภายหลัง

ไฟล์ `ไม่มีไฟล์ใหม่` สร้าง item และ trigger ผ่าน API แล้วผลจะอยู่ใน `lab/ndjson-sink/captured/events/`

ขั้นตอน

```
api '{"jsonrpc":"2.0","method":"host.get","params":{"output":["hostid","host"],
  "filter":{"host":["Zabbix server"]},"id":1}'
HOSTID=<ค่าที่ได้>

api '{"jsonrpc":"2.0","method":"item.create","id":2,"params":{"
  "hostid":"'$HOSTID'", "name":"Lab event probe", "key_":"lab.event.probe",
  "type":2, "value_type":3, "tags":[{"tag":"component", "value":"lab"}]}'
ITEMID=<ค่าที่ได้>

api '{"jsonrpc":"2.0","method":"trigger.create","id":3,"params":{"
  "description":"Lab: event probe is {ITEM.LASTVALUE}",
  "expression":"last(/Zabbix server/lab.event.probe)>=1",
  "priority":2, "tags":[{"tag":"scope", "value":"lab"}]}'

docker compose -f engine/docker-compose.yml exec zabbix-server zabbix_server -R config_cache_reload

# problem
api '{"jsonrpc":"2.0","method":"history.push","params":[{"itemid":"'ITEMID'", "value":"1"}],"id":4}'
# รอประมาณ 10 วินาที แล้ว resolve
api '{"jsonrpc":"2.0","method":"history.push","params":[{"itemid":"'ITEMID'", "value":"0"}],"id":5}'
```

ความหมายของค่าที่ใช้: `type 2` คือ Zabbix trapper, `value_type 3` คือ numeric unsigned, `priority 2` คือ Warning และ `history.push` เป็น API ที่มีตั้งแต่ 7.0

ตรวจสอบว่าผ่าน

```
docker logs netwatch-sink | grep events
# expect: [events] batch 1 (problem) และ [events] batch 2 (recovery)
python -m json.tool lab/ndjson-sink/captured/events/0001.ndjson
python -m json.tool lab/ndjson-sink/captured/events/0002.ndjson
```

ข้อผิดพลาดที่พบบ่อย

- Push ก่อน `config_cache_reload` แล้วได้ error ว่า item ไม่มีอยู่
- Push ค่า 1 เข้า สถานะยังเป็น PROBLEM เหมือนเดิม จึงไม่เกิด event ใหม่
- ลบ probe ทั้งหลังทดสอบ ให้เก็บไว้ เพราะจะใช้ทดสอบ `hooks.py` และ SSE ใน Phase 1

0.7 บันทึกสิ่งที่เรียนรู้ □ ต้องทำ

เป้าหมาย เปลี่ยนสิ่งที่เห็นใน payload จริงให้เป็น spec ที่ใช้อ้างอิงต่อไป

ไฟล์ `docs/connector-payload.md`, `docs/decisions/0002-timescaledb-version-pin.md`, `docs/decisions/0004-`, `docs/decisions/0005-`, `docs/phase-0-checklist.md`

ขั้นตอน

1. `connector-payload.md`: ตารางของทุก field ใน value line และ event line พร้อมชนิดข้อมูลและตัวอย่าง 1 บรรทัดจริงของแต่ละแบบ
2. ระบุให้ชัดว่า value แต่ละ `value_type` มีหน้าตาอย่างไร (float, unsigned, text, log) และ recovery event อ้างอิงกลับไปหา problem event ด้วย field ไหน
3. ADR 0002: บันทึกเหตุผลที่ pin 2.28.3-pg16 กับ Zabbix 7.0.29 (2.29.X ต้องใช้ 7.0.30)
4. ADR 0004 และ 0005: เขียนใหม่ตามที่ตัดสินใจไว้ใน `CLAUDE.md` §8

ตรวจสอบว่าผ่าน

ทุก field ที่เขียนใน `connector-payload.md` หาเจอจริงในไฟล์ `captured/` และไม่มี field ไหนที่ได้มาจาก "เอกสารบอกว่า"

0.8 Git repository □ ต้องทำ

เป้าหมาย เริ่มเก็บประวัติการแก้ไข โดยไม่ให้ secret หลุดเข้า repo

ไฟล์ `.gitignore`, `.gitattributes`

```
# .gitignore
.env
.venv/
__pycache__/
*.egg-info/
*.pyc
lab/ndjson-sink/.sink-token
node_modules/
frontend/dist/
```

ตรวจว่าผ่าน

```
git init && git add -A && git status --short | grep -E '\.env$|sink-token|\.venv'
# expect: ไม่มีผลลัพธ์ (exit 1)
```

ข้อผิดพลาดที่พบบ่อย

- ไม่ได้สร้าง `.gitignore` ก่อน `git add` ถ้า secret ถูก commit ไปแล้ว ลบออกทีหลังก็ไม่พอ ต้อง rotate secret ทุกตัว
- โปรเจกต์อยู่ใน OneDrive ซึ่ง sync ทุกไฟล์รวมถึง `.env` ขึ้น cloud ด้วย ควรพิจารณาย้ายออกจาก OneDrive

0.9 Re-verify ข้อเท็จจริงใน §7 แล้วปิด Phase 0 □ **ต้องทำ**

รันคำสั่งในภาคผนวก A ทุกข้อ บันทึกผลลงใน `docs/phase-0-checklist.md` พร้อมวันที่และ version แล้วจึงเริ่ม Phase 1

4. Phase 1: Backend core

เป้าหมายของ **phase**: ทำ backend ให้ครบทั้ง 2 ทาง ยังไม่ต้องมีอุปกรณ์จริง ให้ใช้ probe จาก 0.6 กับ internal item ของ Zabbix เป็นแหล่งข้อมูล
เงื่อนไขปิด **phase**: push probe เป็น 1 แล้ว problem ต้องโผล่ใน API ของเราและมาทาง SSE ภายในไม่กี่วินาที โดย:

- ไม่มีการเรียก `history.get` เลยสักครั้ง
- `pytest` ผ่านทั้งหมด
- `check-boundary.sh` ได้ OK

ก่อนเริ่ม: ขั้น 0.4 ต้องเสร็จแล้ว เพราะไฟล์แรกของ phase นี้อยู่ใน guarded path

1.1 Domain: models, enums, ports

เป้าหมาย นิยามภาษาของ product เอง โดยไม่ผูกกับ engine

ไฟล์ `backend/app/domain/models.py`, `enums.py`, `ports.py`

รายละเอียด

- **models.py**: `Device` (`ref`, `name`, `kind`, `site`, `status`), `MetricPoint` (`device_ref`, `metric`, `ts`, `value`), `Problem` (`ref`, `device_ref`, `severity`, `title`, `started_at`, `resolved_at`) เวลาให้ใช้ `datetime` แบบมี `timezone` ห้ามใช้คู่ `clock / ns`
- **enums.py**: `Severity` และ `DeviceStatus` เป็นของเราเอง การแปลงค่าจาก engine ทำใน mapper เท่านั้น
- **ports.py**: `MonitoringEnginePort` เป็น `typing.Protocol` มีเฉพาะงาน `config/lookup` เช่น `list_devices(token)`, `get_device(ref, token)` และ **ไม่มี method ที่อ่าน history**

ตรวจว่าผ่าน

```
bash ops/check-boundary.sh ต้องได้ OK และ python -c "import app.domain.ports" ต้องไม่มี error
```

ข้อผิดพลาดที่พบบ่อย

- ตั้งชื่อ field เลียนแบบ engine เช่น `hostid` หรือ `clock`
- ใส่ method `get_history` ลงใน port ซึ่งเปิดทางให้ละเมิด §3

1.2 Connector schema

เป้าหมาย แปลง NDJSON จาก engine ให้เป็น object ที่ validate แล้ว โดยเขียนจาก payload จริงที่เก็บไว้ใน 0.5 และ 0.6

ไฟล์ `backend/app/zabbix/connector_schema.py`, `backend/tests/fixtures/connector/*.ndjson` (copy มาจาก `captured/`), `backend/tests/test_connector_schema.py`

รายละเอียด

- ใช้ `Pydantic model` 1 ตัวสำหรับ `value line` และ 1 ตัวสำหรับ `event line` โดยตั้ง `extra="ignore"` ไว้เพื่อให้ field ใหม่จาก patch release ไม่ทำให้ระบบพัง
- Parse ที่ละบรรทัด ถ้าบรรทัดไหนเสียให้ log แล้วข้ามไป ห้ามทิ้งทั้ง `batch`
- Test ต้อง parse fixture จริงได้ครบทุกบรรทัด และมีเคสบรรทัดเสียด้วย

ตรวจว่าผ่าน

```
cd backend && .venv\Scripts\python -m pytest tests/test_connector_schema.py -q
# expect: all passed
```

1.3 Mapper, adapter และ connector ingest

เป้าหมาย เป็นจุดเดียวที่แปลงระหว่างโลกของ engine กับ domain ของเรา

ไฟล์ `backend/app/zabbix/mapper.py`, `adapter.py`, `connector_ingest.py`

รายละเอียด

- **mapper.py**: `device_from_api()`, `problem_from_event()` และฟังก์ชันสร้าง/แยก `ref` (`opaque string`) ต้องไม่มีโค้ดส่วนอื่นนอกจาก `mapper` ที่ parse `ref` ได้ และต้อง **ไม่สนใจ** `lastvalue` (ADR 0005)
- **adapter.py**: class ที่ implement `MonitoringEnginePort` โดยเรียก `client.py` และรับ token จากผู้เรียกทุกครั้ง

- **connector_ingest.py**: รับ object จาก schema แล้วคืนค่าเป็น MetricPoint หรือ Problem ของ domain

ตรวจว่าผ่าน

Unit test ของ mapper ต้องแปลงไป-กลับได้ (ref → id → ref) และ test ของ adapter ใช้ `httpx.MockTransport` ผ่าน parameter `transport=` ของ `ZabbixClient`

1.4 Store ของเราเอง

เป้าหมาย เป็นที่เก็บ time-series และ problem ที่กราฟและ tile ทุกตัวอ่าน

ไฟล์ `backend/app/store/sqlite_store.py` (ใช้กับ demo และ test), `backend/app/store/postgres_store.py` (ใช้กับ appliance)

รายละเอียด

- Table หลัก: `devices`, `metric_points` (เป็น hypertable บน TimescaleDB), `problems` และ `latest_values` (ตาม ADR 0005)
- ใช้ **database แยก** (เช่น `acme`) พร้อม role แยก (`acme_app`) และ role นี้ต้องไม่มีสิทธิ์ใน DB `zabbix`
- Unique key ของ metric คือ (`device_ref`, `metric`, `ts`) และเขียนแบบ upsert เพราะ Connector อาจ retry ส่งซ้ำได้สูงสุด 5 ครั้ง
- Downsampling ทำใน store ด้วย `time_bucket()` ห้ามทำใน API ของ engine

ตรวจว่าผ่าน

```
docker compose -f engine/docker-compose.yml exec db psql -U acme_app -d zabbix -c "select 1"
# expect: permission denied / FATAL ซึ่งแสดงว่า role ของเราแตะ DB ของ engine ไม่ได้
```

ข้อผิดพลาดที่พบบ่อย

- ใช้ user `zabbix` กับ store ของเราด้วย ทำให้ไม่มีการแยกกันจริง
- ไม่มี unique key ข้อมูลจะซ้ำทุกครั้งที่มีการ retry

1.5 api/hooks.py : ตัวรับ push จริง

เป้าหมาย แทนที่ sink ใน lab ด้วย endpoint ของ backend

ไฟล์ `backend/app/api/hooks.py`, `backend/app/services/ingest.py`

รายละเอียด

- Endpoint `POST /hooks/values` และ `POST /hooks/events` ตรวจ Bearer token แล้วอ่าน body ส่งต่อให้ ingest และตอบ 200 ให้เร็วที่สุด
- `hooks.py` ทำหน้าที่แค่ส่งต่อข้อมูล (transport) **ไม่มีคำว่า engine และไม่ parse schema** เอง โดยรับ parser ผ่าน port ที่ `container.py` ผูกเข้ามา
- ใช้ `connector.update` ย้าย URL จาก `netwatch-sink` มาที่ container ของ backend ซึ่งต้องอยู่ใน network เดียวกัน

ตรวจว่าผ่าน

Push probe เป็น 1 แล้ว `problems` ใน store ต้องมี row ใหม่ และ log ของ backend ต้องเห็น batch ของ values เข้ามาทุกนาที่

1.6 Read services + API endpoints

ไฟล์ `backend/app/services/inventory.py`, `metrics.py`, `problems.py`, `backend/app/api/*.py`

- `GET /api/devices`, `GET /api/devices/{ref}`, `GET /api/devices/{ref}/metrics?from=&to=&step=`, `GET /api/problems`
- Metric และ latest value ทั้งหมดอ่านจาก store ส่วนรายชื่ออุปกรณ์และ config อ่านผ่าน port แล้ว cache ไว้

ตรวจว่าผ่าน

เปิด log ของ web container ของ engine ไว้ระหว่างเรียก `/metrics` ต้องไม่มี request ใดๆ เข้าไปที่ engine

1.7 Multi-tenant และ auth

ไฟล์ `backend/app/auth/`, ส่วนที่เกี่ยวข้องใน `provisioning/provision.py`

- ความสัมพันธ์คือ user → tenant → host group
- แต่ละ tenant มี user และ API token ของตัวเองใน engine (ADR 0004) และ read path ใช้ token ของ tenant นั้น
- ข้อมูลจาก store ต้องกรองด้วยชุด `device_ref` ของ tenant ด้วยเสมอ เพราะสิทธิ์ใน engine คุ่มครองเฉพาะทาง API ไม่ครอบคลุม store ของเรา

- Token ของ tenant เก็บแบบเข้ารหัส ห้ามเก็บใน git

ตรวจว่าผ่าน

`provision.py` พิมพ์ PASS ของแต่ละ tenant ออกมา คือ tenant A เห็นเฉพาะ host ของตัวเองครบทุกตัว และ tenant B เห็นแค่ของ B เมื่อเรียก API ของเราด้วย user ของ A ต้องไม่เห็นข้อมูลของ B เลย

Decision ใหม่ที่ต้องปิด: วิธี login เข้า portal ของเราเอง (session cookie หรือ JWT, มี SSO หรือไม่)

1.8 Live updates: bus, SSE และ cache

ไฟล์

`backend/app/bus/` (memory และ Redis), `backend/app/services/stream.py`, `backend/app/cache.py`

- Ingest publish ข้อมูลเข้า bus แล้ว SSE endpoint `/api/stream` subscribe โดยกรองตาม tenant
- ถ้ามีหลาย uvicorn worker ต้องใช้ Redis pub/sub เพราะ bus แบบ memory จะส่งได้เฉพาะภายใน worker ตัวเดียว
- `cache.py` cache response จาก API ของ engine โดยใช้ key ที่รวม hash ของ token ของ tenant ด้วย เพื่อกันข้อมูลข้าม tenant

ตรวจว่าผ่าน

```
curl -N -H "Authorization: ..." http://127.0.0.1:8000/api/stream
# จากนั้น push probe = 1 → ต้องเห็น event problem ภายในไม่กี่วินาที
```

1.9 Wiring และ test

ไฟล์

`backend/app/container.py`, `backend/app/main.py`, `backend/tests/`

- `container.py` เป็นจุดเดียวที่ผูก adapter, store และ bus เข้ากับ services และเป็น 1 ใน 2 ที่ได้รับยกเว้นจากกฎ boundary
- มี test ใน pytest ที่ตรวจกฎ boundary เข้ากับ script ด้วย จะได้ทำงานใน CI แม้ไม่มี bash
- `main.py` serve portal ที่ build แล้ว (Phase 3)

5. Phase 2: Provisioning และอุปกรณ์จริง

เป้าหมายของ phase: สร้างและแก้ host, template และ tenant ใน engine ทั้งหมดด้วยโค้ด แล้วนำอุปกรณ์จริงเข้าระบบทีละตัว

เงื่อนไขปิด phase:

- อุปกรณ์ทุกตัวใน `inventory.csv` ส่งค่าเข้า store ของเรา
- ค่า throughput ตรงกับ `show interfaces`
- ปิด core แล้วต้องได้ alert เดียว ไม่ใช่ alert จากอุปกรณ์ปลายทาง N ตัว

2.1 Export template ต้นฉบับ

ไฟล์ `templates/upstream/*.yaml` (ห้ามแก้ไข และมี deny rule ใน settings กำกับอยู่)

- หา template ด้วย `template.get` โดยใช้ `search` เช่น "Cisco IOS", "Aruba" แล้ว export ด้วย `configuration.export` ในรูปแบบ YAML
- ชื่อ template ขึ้นกับ version ของ Zabbix ให้ใช้ชื่อตามที่ instance ของเราแสดง

2.2 Template ของเรา

ไฟล์ `templates/acme/*.yaml` (export ด้วย `provisioning/export.py`)

- Clone** จาก upstream ก่อนแก้ไข
- จัดโครงสร้างเป็น master item (เช่น SNMP walk หรือ HTTP agent) + dependent item ที่ใช้ JSONPath หรือ preprocessing เพื่อรองรับทั้ง 2 ทางเลือกของ Aruba ใน decision #1
- Interface ใช้ LLD ร่วมกับ `ifHCInOctets/ifHCOutOctets` ที่ poll ทุก 60 วินาที ส่วน error และ discard ให้ poll ซ้ำลงได้
- ติด tag ให้ทุก item เช่น `component` และ `tenant` ไว้ใช้กรอง Connector ในอนาคต

Blocked: ส่วนของ Aruba รอ decision #1 (ขอ PO จากลูกค้า) ฝั่ง Cisco เริ่มได้ก่อน

2.3 Provisioning scripts

ไฟล์ `provisioning/inventory.csv`, `catalog.py`, `provision.py`, `verify.py`, `export.py`

ไฟล์	หน้าที่
<code>inventory.csv</code>	คอลัมน์ name, ip, site, role, vendor, model, tenant (ห้ามมี credential)
<code>catalog.py</code>	map role + model ไปเป็น template และ macro
<code>provision.py</code>	idempotent: สร้างหรืออัปเดต host group, host, tenant user, tenant token และ connector รันซ้ำแล้วต้องไม่มีอะไรเปลี่ยน และต้องรันการตรวจสอบตาม ADR 0004 ทุกครั้ง
<code>verify.py</code>	เทียบสถานะใน engine กับ inventory แล้วรายงานส่วนที่ไม่ตรงกัน
<code>export.py</code>	export template ของเราลง <code>templates/acme/</code> เพื่อให้ diff ได้ใน git

ตรวจว่าผ่าน

รัน `provision.py` 2 รอบติดกัน รอบที่ 2 ต้องรายงานว่า 0 changes และทุก tenant ได้ PASS

2.4 อุปกรณ์ตัวแรก: Cisco core switch

Config บนตัวอุปกรณ์ (ตัวอย่าง ให้ตรวจ syntax กับ IOS version จริง)

```
snmp-server view ACME-RO iso included
snmp-server group ACME-RO v3 priv read ACME-RO access 99
snmp-server user acme-nms ACME-RO v3 auth sha <AUTH-PASS> priv aes 128 <PRIV-PASS>
snmp-server ifindex persist
access-list 99 permit <IP ของเครื่อง docker host>
```

- Credential ของ SNMPv3 อยู่ใน Secret text macro หรือ Vault macro ของ engine เท่านั้น
- ACL ต้องอนุญาต IP ของ **docker host** เพราะ SNMP ออกไปผ่าน NAT ของ Docker

ตรวจว่าผ่าน

- ใช้ปุ่ม "Test" ของ item ใน engine หรือ `snmpget` จาก tools container ต้องได้ค่า
- Throughput ใน store ต้องตรงกับ `show interfaces` โดยคลาดเคลื่อนได้ไม่เกินรอบ poll
- `show snmp user` ต้องแสดง auth และ priv ถูกต้อง

ข้อผิดพลาดที่พบบ่อย

- ลืม `ifindex persist` พอ reboot แล้ว `ifIndex` จะเลื่อน ทำให้กราฟสลับ interface กัน
- ใช้ counter 32-bit กับลิงก์ 1G ขึ้นไป counter จะ wrap ภายในไม่กี่วินาที
- เปิด trap ซึ่งยังไม่ได้ตัดสินใจ (§2)

2.5 Rollout: access switch และ AP

- เพิ่มอุปกรณ์ที่ละกลุ่มผ่าน `inventory.csv` แล้วรัน `provision.py`
- ตั้ง dependent trigger ให้ "access unreachable" ขึ้นกับ "core unreachable" และ "AP down" ขึ้นกับ uplink ของ access switch
- Decision #3 (proxy ที่ site หรือ poll ตรง) ต้องปิดก่อนขั้นนี้ ถ้าลูกค้าอยู่หลาย site

ตรวจว่าผ่าน

Shutdown uplink ของ core ใน maintenance window แล้วต้องได้ problem เดียว ไม่ใช่ problem ของ access switch ทุกตัว

2.6 Demo network

ไฟล์

`lab/demo/feeder.py`

ป้อนค่าจำลองผ่าน engine จริง (trapper + `history.push`) เพื่อให้ demo ไปได้โดยไม่ต้องมีอุปกรณ์ ข้อมูลยังต้องไหลผ่าน Connector เหมือนของจริง

6. Phase 3: Frontend

เป้าหมายของ **phase**: portal ที่เขียนเองทั้งหมด ไม่มีโค้ด CSS, theme หรือ view จาก Zabbix

เงื่อนไขปิด **phase**: ลูกค้า login แล้วเห็น overview, รายการอุปกรณ์, กราฟ และ problem แบบ live และ `check-boundary.sh` ได้ OK เพราะ `frontend/` ก็อยู่ใน guarded path

3.1 โครงโปรเจกต์

- `frontend/` : React + Vite + TypeScript
- UI kit ใช้ได้เฉพาะ license MIT, BSD หรือ Apache เช่น MUI, Ant Design หรือ shadcn/ui + Tailwind
- ใช้ API client ตัวเดียว และใช้ type ที่ตรงกับ response ของ backend (`device_ref` , `problem_ref`)

3.2 หน้าจอ

หน้า	ข้อมูลมาจาก	หมายเหตุ
Login	auth ของเรา	ตาม decision เรื่องวิธี login ใน 1.7
Overview	<code>/api/devices</code> + SSE	tile สถานะและ latest value จาก store
Device list	<code>/api/devices</code>	กรองตาม site และ role
Device detail	<code>/api/devices/{ref}/metrics</code>	กราฟ SVG ที่เขียนเอง ใช้ข้อมูลชุดเดียวกับ tile
Problems	<code>/api/problems</code> + SSE	เรียงตาม severity และเวลา

3.3 กราฟ SVG

- เขียน component line/area chart เอง โดยรับข้อมูลที่ downsample มาแล้วจาก backend
- ส่ง `step` ตามความกว้างของกราฟ อย่างถึงจุดข้อมูลดิบทั้งหมดมา

3.4 Build และ serve

`npm run build` แล้ว `main.py` ของ backend จะ serve `frontend/dist/`

ข้อผิดพลาดที่พบบ่อย

- ตั้งชื่อตัวแปรหรือ CSS class ว่า `zbx-*` ซึ่ง guard จะจับได้
- เรียก API ของ engine ตรงจาก browser ซึ่งผิดทั้ง §3 และ §4
- ใช้ library กราฟที่มี license ไม่ผ่านเงื่อนไข

7. Phase 4: ส่งมอบ appliance

เป้าหมายของ **phase**: ติดตั้ง ดูแล backup/restore และส่งมอบให้ลูกค้าได้ โดยปฏิบัติตาม license ครบถ้วน

เงื่อนไขปิด **phase**:

- ติดตั้งจาก 0 บนเครื่องใหม่ได้ตาม runbook
- Restore จาก backup แล้วข้อมูลกลับมาครบ
- เอกสาร license ครบ

งาน	ไฟล์	รายละเอียดและวิธีตรวจ
Compose ที่ใช้ส่งมอบ	ops/docker-compose.acme.yml	รวม engine + backend + Redis + reverse proxy (TLS) เปิดเฉพาะ port 443 ส่วนอื่นอยู่ใน network ภายใน
Health check	ops/healthcheck	ตรวจทุก container, คิวของ Connector (zabbix[connector_queue]) และเวลาที่ batch ล่าสุดเข้ามา ถ้าไม่มีข้อมูลเข้ามาเกิน 5 นาทีต้องแจ้งเตือน
Backup	ops/backup	pg_dump ทั้ง DB ของ engine และของเรา พร้อม templates/acme/ และกำหนด retention
Restore	ops/restore	ข้อม restore จরিบนเครื่องเปล่าอย่างน้อย 1 ครั้งก่อนส่งมอบ
การแจ้งเตือน	templates/media/line_webhook.js , docs/alert-catalog.md	media type แบบ webhook ไปที่ LINE และระบุใน catalog ว่า alert ไหนส่งถึงใคร ที่ severity ระดับใด
เอกสาร	docs/runbook.md , docs/architecture.md , docs/demo-script.md	ให้คนที่ไม่เคยเห็นระบบลองติดตั้งตาม runbook
Hardening	-	เปลี่ยนรหัส Admin, ตั้งวันหมดอายุของ token, ปิด user ที่ไม่ใช่ และไม่เปิด port 8080 ออกนอกเครื่อง
License	LICENSES/	แนบ LICENSE ของ Zabbix (AGPLv3) และ written offer for source ห้ามลบ copyright notice และแนบ license ของ dependency ทั้งหมด

ข้อผิดพลาดที่พบบ่อย

- Backup แค่ DB ของ engine แต่ลืม store ของเรา กราฟย้อนหลังจะหายทั้งหมด
- ไม่เคยลอง restore จริง แล้วมารู้ตอนเกิดเหตุว่า backup ใช้ไม่ได้
- ส่งมอบโดยไม่แนบ source offer ของ Zabbix ซึ่งผิด AGPLv3

8. Decision ที่ยังเปิดอยู่

#	เรื่อง	ต้องปิดก่อน	สถานะและสิ่งที่ต้องทำ
1	Aruba AOS-8 (SNMP ไปที่ controller) หรือ AOS-10 + Central (REST, OAuth2)	2.2	OPEN: ขอ PO จากลูกค้าว่าซื้อ AP รุ่นไหนและ license อะไร ระหว่างรอให้ออกแบบ template แบบ master + dependent
2	TimescaleDB version pin	0.7	ตัดสินใจแล้ว: 2.28.3-pg16 เหลือแค่เขียน ADR 0002
3	Proxy ที่ site หรือ poll ตรง	2.5	OPEN: ขึ้นกับ network design และจำนวน site
4	Multi-tenant token	-	ตัดสินใจแล้ว: token แยกต่อ tenant (ADR 0004)
5	แหล่งของ latest value	-	ตัดสินใจแล้ว: อ่านจาก store ของเรา (ADR 0005)
ใหม่	วิธี login เข้า portal	1.7	OPEN: session หรือ JWT และมี SSO หรือไม่
ใหม่	ระยะเวลาเก็บข้อมูลใน store	1.4	OPEN: เก็บข้อมูลดิบกี่วัน และข้อมูล downsample ก็เดือน
ใหม่	Store อยู่ใน Postgres ตัวเดียวกับ engine (คนละ DB) หรือแยก container	1.4	OPEN: แนะนำให้เริ่มจาก DB แยกใน instance เดียวกัน แล้วแยก container ตอนทำ appliance ถ้าโหลดสูง

ภาคผนวก A: คำสั่ง re-verify ข้อเท็จจริงของ Zabbix

ต้องรันใหม่ทุกครั้งที่เปลี่ยน image tag แล้วบันทึกผลลงใน `docs/phase-0-checklist.md`

A.1 License

```
docker run --rm zabbix/zabbix-server-pgsql:alpine-7.0.29 zabbix_server -V
# expect: copyright line ที่ระบุ GNU Affero General Public License v3
```

A.2 TimescaleDB อยู่ในช่วงที่รองรับ

```
docker compose -f engine/docker-compose.yml exec -T db psql -U zabbix -d zabbix \
-c "SELECT extversion FROM pg_extension WHERE extname='timescaledb';"
docker compose -f engine/docker-compose.yml logs zabbix-server | grep -i timescale
# expect: 2.28.3 และไม่มีบรรทัดที่มีคำว่า "unsupported"
```

A.3 item.get คืนค่า lastvalue หรือไม่

```
curl -s -X POST http://127.0.0.1:8080/api_jsonrpc.php \
-H 'Content-Type: application/json-rpc' \
-H "Authorization: Bearer $ENGINE_TOKEN" \
-d '{"jsonrpc":"2.0","method":"item.get","params":{"limit":1,"output":"extend"},"id":1}' \
| python -c "import sys,json; print(sorted(json.load(sys.stdin)['result'][0].keys()))"
# expect: มี lastvalue, lastclock, prevvalue
# (ถึงจะมี ก็ไม่ใช่ตาม ADR 0005)
```

A.4 Connector worker ทำงานอยู่

```
docker compose -f engine/docker-compose.yml top zabbix-server | grep -i connector
# expect: connector manager #1 และ connector worker ตามค่าของ ZBX_STARTCONNECTORS
```

A.5 Smoke test ของ API client

```
cd backend && .venv\Scripts\python -m app.zabbix.smoke
```

ภาคผนวก B: ops/check-boundary.sh

บันทึกเป็น LF แล้วรันด้วย `bash ops/check-boundary.sh`

```
#!/usr/bin/env bash
# ops/check-boundary.sh - enforce CLAUDE.md §4.
#
# Fails if an engine-specific word appears in the guarded paths. The only
# places allowed to know the engine are backend/app/zabbix/** and
# backend/app/container.py - neither is scanned here.
#
# Runs in CI on every commit, and as a PostToolUse hook on every Write/Edit.
# Exit 0 = clean. Exit 2 = violation (Claude Code feeds stderr back to the model).

set -euo pipefail

cd "$(dirname "$0")/.."

GUARDED=(
    backend/app/domain
    backend/app/services
    backend/app/api
    backend/app/auth
    frontend
)

EXCLUDES=(
    --exclude-dir=node_modules
    --exclude-dir=dist
    --exclude-dir=.venv
    --exclude-dir=__pycache__
    '--exclude=*.pyc'
)

# Paths are rebuilt one step at a time - skip the ones that do not exist yet.
targets=()
for d in "${GUARDED[@]%%/*}"; do
    [ -e "$d" ] && targets+=("$d")
done

if [ ${#targets[@]} -eq 0 ]; then
    echo "check-boundary: no guarded paths exist yet - OK"
    exit 0
fi

found=0

# $1 = label, rest = grep pattern args
report() {
    local label=$1; shift
    local hits
    if hits=$(grep -rni "${EXCLUDES[@]}" "$@" "${targets[@]}"); then
        echo "BOUNDARY VIOLATION [$label]:" >&2
        echo "$hits" >&2
        found=1
    fi
}

# Engine name, any case, anywhere in a word: ZabbixClient, ZABBIX_URL, ...
report "engine name" -i -e 'zabbix'

# Engine IDs. "_" counts as a separator, so device_hostid and hostids are caught.
report "engine id" -iE -e '([a-z0-9])(event|host|item|trigger)ids?([a-z0-9]|$)'

# Short field names: whole word only, otherwise "ns" matches half the codebase.
report "engine field" -w -e 'clock' -e 'ns'

if [ "$found" -ne 0 ]; then
```

```
    echo "See CLAUDE.md §4: use device_ref / problem_ref; only zabbix/mapper.py knows the real IDs." >&2
    exit 2
fi

echo "check-boundary: OK (${targets[*]})"
```

เป็นเครื่องมือใน lab เท่านั้น ห้ามย้ายเข้าไปใน backend/app/ เพราะตัวรับจริงคือ api/hooks.py ใน Phase 1

```

"""Phase-0 receiver for the engine's Connector push (CLAUDE.md §3).

Lab tool, not product code. It proves the push path works end to end and
keeps the first raw batches on disk, so connector_schema.py is written from
real payloads instead of from the docs.

Runs inside the engine's Docker network. Stdlib only, so the container
needs nothing installed.
"""

from __future__ import annotations

import os
import re
from collections import Counter
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from threading import Lock

TOKEN = os.environ["SINK_TOKEN"]
PORT = int(os.environ.get("SINK_PORT", "8099"))
KEEP = int(os.environ.get("SINK_KEEP", "20")) # batches saved per path; the rest are only counted
OUT = Path(__file__).resolve().parent / "captured"

_batches: Counter[str] = Counter()
_lock = Lock()

def _read_body(handler: BaseHTTPRequestHandler) -> bytes:
    length = handler.headers.get("Content-Length")
    if length is not None:
        return handler.rfile.read(int(length))
    # Transfer-Encoding: chunked
    body = bytearray()
    while True:
        size = int(handler.rfile.readline().split(b";")[0].strip(), 16)
        if size == 0:
            handler.rfile.readline()
            return bytes(body)
        body += handler.rfile.read(size)
        handler.rfile.readline()

class Handler(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"

    def do_POST(self) -> None:
        body = _read_body(self)

        if self.headers.get("Authorization") != f"Bearer {TOKEN}":
            print(f"401 {self.path}: bad or missing Bearer token", flush=True)
            self._reply(401)
            return

        name = re.sub(r"^[A-Za-z0-9_-]", "_", self.path.strip("/")) or "root"
        with _lock:
            _batches[name] += 1
            n = _batches[name]

        lines = len(body.splitlines())
        print(
            f"[{name}] batch {n}: {lines} lines, {len(body)} bytes, "
            f"Content-Type={self.headers.get('Content-Type')}",
            flush=True,

```

```

    )

    if n <= KEEP:
        folder = OUT / name
        folder.mkdir(parents=True, exist_ok=True)
        (folder / f"{n:04d}.ndjson").write_bytes(body)

    self._reply(200)

def _reply(self, status: int) -> None:
    self.send_response(status)
    self.send_header("Content-Length", "0")
    self.end_headers()

def log_message(self, format: str, *args: object) -> None:
    pass # the one line per batch above is enough

if __name__ == "__main__":
    print(f"ndjson-sink on :{PORT}, saving first {KEEP} batches per path to {OUT}", flush=True)
    ThreadingHTTPServer(("0.0.0.0", PORT), Handler).serve_forever()

```